

Module 3: Syntactic Parsing & Grammars

CFG, Chomsky Normal Form (CNF), CYK Dynamic Parsing Trace & Dependency Transition Systems

Module III: Syntactic Parsing, Context-Free Grammars & Dependency Structures

1. Phrase Structure Grammars & Constituency vs. Dependency Syntax Trees
2. Context-Free Grammars (CFG) for Natural Language Syntax ($G = (V, T, P, S)$)
3. Structural Ambiguity in Natural Language (Prepositional Phrase & Coordination)
4. Chomsky Normal Form (CNF) Formal Conversion Rules ($A \rightarrow BC \mid a$)
5. The CYK (Cocke-Younger-Kasami) Dynamic Programming Parsing Algorithm
6. Complete Step-by-Step CYK Triangular Table Parsing Trace for "Book the flight"
7. Probabilistic Context-Free Grammars (PCFG) & Maximum Likelihood Estimation
8. The Probabilistic CYK (Inside-Outside) Parsing Algorithm for Disambiguation
9. Dependency Grammar Foundations: Heads, Dependents & Universal Dependencies (UD)
10. Transition-Based Dependency Parsing (Shift-Reduce, ARC-STANDARD & ARC-EAGER)
11. Graph-Based Dependency Parsing (Chu-Liu-Edmonds Maximum Spanning Tree)
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 4 & 5: Chomsky Normal Form (CNF) & The CYK Algorithm

A Context-Free Grammar is in **Chomsky Normal Form (CNF)** if every production rule has one of two forms:

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a$$

Where A, B, C are non-terminals ($B, C \neq S$) and a is a single terminal symbol.

CYK Dynamic Programming Recurrence: Let table entry $P[i, j, A]$ be true if non-terminal A derives substring $w_i \dots w_j$ of length $l = j - i + 1$. **Base Case (Length $l = 1$):**

$$P[i, i, A] = \text{true} \quad \text{for all } A \rightarrow w_i \in P$$

2. Inductive Step (Length $l = 2 \dots n$):

$$P[i, j, A] = \text{true} \iff \exists k \in [i, j-1] \text{ and } A \rightarrow BC \in P \text{ such that } P[i, k, B] = \text{true} \wedge P[k+1, j, C] = \text{true}$$

Time Complexity: $O(n^3 \cdot |P|)$ where n is sentence length and $|P|$ is number of CNF grammar rules.

Topic 9 & 10: Dependency Parsing & Transition Systems

Dependency Syntax describes grammatical structure by establishing direct binary asymmetric grammatical relations between words (Heads $\rightarrow$ Dependents), without intermediate phrase nodes:

Transition Action	Stack & Buffer Manipulation	Dependency Arc Created
1. SHIFT	Remove first word w from Buffer and push onto Stack.	None.
2. LEFT-ARC (r)	Creates dependency arc $s_1 \xleftarrow{r} s_2$ from top of stack s_1 to second item s_2 . Pops s_2 from stack.	$s_1 \rightarrow s_2$ (s_1 is Head).
3. RIGHT-ARC (r)	Creates dependency arc $s_1 \xrightarrow{r} s_2$ from second item s_1 to top of stack s_2 . Pops s_2 from stack.	$s_1 \rightarrow s_2$ (s_1 is Head).

Q1. State the Chomsky Normal Form (CNF) restrictions and explain why CNF is mandatory for the CYK algorithm. (8 Marks)

- **CNF Restrictions:** All productions must strictly be of the form $A \rightarrow BC$ (exactly 2 non-terminals) or $A \rightarrow a$ (exactly 1 terminal). No unit productions ($A \rightarrow B$) or ϵ -productions ($A \rightarrow \epsilon$) are allowed.
- **Why Mandatory for CYK:** CYK relies on dynamic programming by splitting a substring into exactly two parts ($w[i \dots k]$ and $w[k + 1 \dots j]$). Binary branching ($A \rightarrow BC$) guarantees that any subphrase is formed by joining exactly two adjacent subphrases, creating the $O(n^3)$ tabular recurrence.